

COMMON COURSE ASSIGNMENT

CSA65 – GENERATIVE AI AND LARGE LANGUAGE MODELS | Covering CO2, CO3 & CO4 (Units II, III & IV)

Assignment Information

Department	Computer Science and Engineering
Programme	B.E. / B.Tech – CSE
Course Code & Course Name	CSA65 – Generative AI and Large Language Models
Academic Year / Batch	2026 – 2027
Faculty Name	Dr. Malathi.k
Assignment Title	Enterprise Knowledge Assistant Using Retrieval-Augmented Generation and AI Agents
Date of Issue	02.09.2026
Date of Submission	03.19.2026
Maximum Marks	100
Course Outcome(s) – CO	CO2, CO3, CO4
Bloom's Taxonomy Level	L3 – Apply, L4 – Analyze, L5 – Evaluate
SDG Mapping (SDG 1–17)	SDG 9 – Industry, Innovation and Infrastructure
Industry / Societal Relevance	Retrieval-augmented, agent-based knowledge assistants are being adopted across IT services, manufacturing, healthcare and banking to make large internal document repositories searchable, auditable and usable by every employee.

Assignment Problem / Challenge

This assignment requires students to apply core Generative AI concepts — prompt engineering and LLM behaviour (Unit II), retrieval-augmented generation and AI agents (Unit III), and multimodal generative AI and intelligent applications (Unit IV) — to a realistic enterprise deployment problem. Students must analyze the given scenario, apply relevant techniques and models, use modern tools for implementation, validate their results with measurable metrics, and justify their engineering decisions.

Problem Statement

Scenario: Enterprise Knowledge Assistant for "KnowledgeDesk" – An Internal RAG and Agent Platform for a Large Engineering Services Firm

A 4,000-employee engineering services company is deploying an in-house knowledge assistant, "KnowledgeDesk", so that employees can ask natural-language questions about HR policies, quality SOPs, product manuals, past project reports and compliance documents instead of searching a shared drive. The corpus contains about 12,000 documents of roughly 8 pages each, in PDF, DOCX and scanned-image form, and grows by around 400 documents every month. Every answer shown to an employee must cite the source document and page, and the assistant must refuse to answer when the retrieved evidence is insufficient.

As the AI engineering team, the following sections propose and analyze solutions across three core areas of the generative-AI system that will power this assistant: (1) Prompt Engineering & LLM Behaviour, (2) Retrieval-Augmented Generation & AI Agents, and (3) Multimodal GenAI & Intelligent Applications.

Requirements and Constraints

- **Functional:** Answer natural-language questions over the 12,000-document corpus with an inline citation (document name and page) for every factual statement; support 500 concurrent employees.
- **Performance:** End-to-end response time must not exceed 3 seconds for 90% of queries; retrieval Recall@5 on the evaluation set must be at least 0.85.
- **Technical constraint:** The chosen LLM has an 8,192-token context window; the embedding model produces 768-dimensional vectors; one GPU server with 24 GB VRAM is available for local inference.
- **Reliability:** The assistant must not fabricate content — it must abstain and say so when retrieval confidence falls below a stated threshold; groundedness of accepted answers must be at least 90%.
- **Resource constraint:** Monthly inference budget of Rs. 40,000 for hosted API calls; the vector index and document store must fit within 100 GB of disk.
- **Cost & Time:** Solution should use open-source tools and freely available model APIs; project must be completed within the semester timeline.
- **Security/Privacy:** Retrieval must respect role-based access — an employee must never receive content from a department they are not authorised for; personally identifiable information must not be sent to external APIs.
- **Standards:** Follow responsible-AI practice — disclose AI generation, log every query with its retrieved sources, and keep a human escalation path.

Student Work

1. Problem Understanding and Formulation

KnowledgeDesk is an internal, retrieval-augmented, agent-assisted question-answering system for a 4,000-employee engineering services firm. It replaces manual search across a growing ~12,000-document repository (HR policies, quality SOPs, product manuals, project reports, compliance documents) with natural-language Q&A, and must cite its sources on every answer.

Core challenges implied by the scenario

- Prompting: obtaining grounded, consistently formatted, citation-bearing answers from an 8,192-token base LLM without fine-tuning, and making it refuse when evidence is insufficient.
- Retrieval: chunking, embedding and indexing a heterogeneous 12,000-document corpus (PDF, DOCX, scanned images) that grows ~400 documents/month, while staying inside the 100 GB storage budget.
- Agent orchestration: handling multi-step requests (e.g., "compare the current leave policy with last year's version and raise a ticket if it changed") that a single retrieval call cannot satisfy.
- Multimodal handling: extracting and representing scanned drawings, tables and screenshots so they remain retrievable and are not silently dropped.
- Delivery: packaging the system as a responsive application within a 3-second (P90) latency budget and a Rs. 40,000/month inference budget.

Expected outcomes

- Grounded answers with correct inline citations (document + page) for every factual claim.
- Retrieval Recall@5 ≥ 0.85 on a held-out evaluation set.
- Correct tool selection and routing by the agent for multi-step queries.
- End-to-end response time ≤ 3 s for 90% of queries.
- Groundedness of accepted answers $\geq 90\%$, with correct abstention on out-of-corpus or low-confidence queries.

Assumptions made (stated and justified)

- Average 500 tokens/page and 8 pages/document (given) — typical for policy/technical prose; used to size the corpus in tokens.
- Chunk size 512 tokens with 50-token overlap (given) — balances retrieval granularity against index size and preserves context across chunk boundaries.
- Embedding dimensionality 768, float32 (given) — consistent with common open-source sentence-embedding models (e.g., an MPNet/BGE-base-class model) that fit comfortably on a 24 GB GPU.
- Top-k = 12 retrieved chunks per query — derived from the context-window budget in Section 2, Part I, Q2 below.
- Token pricing of Rs. 0.18/1,000 input tokens and Rs. 0.55/1,000 output tokens, and 20,000 queries/month ($\approx 4,000$ employees generating roughly one query every two working days) — used for the cost projection against the Rs. 40,000 budget.
- The monthly growth of ~400 documents is handled by incremental re-indexing of new/changed documents only, not a full corpus rebuild.

Constraints from Section D that the design must satisfy

Functional (citations + 500 concurrent users), Performance (3 s P90, Recall@5 $\geq$ 0.85), Technical (8,192-token context, 768-d embeddings, single 24 GB GPU), Reliability (abstain below confidence threshold, $\geq$ 90% groundedness), Resource (Rs. 40,000/month, $\leq$ 100 GB storage), Cost & Time (open-source/free-tier tools, semester timeline), Security/Privacy (RBAC, no PII to external APIs), and Standards (AI disclosure, query logging, human escalation) — each is addressed explicitly in the design and validation sections below.

2. Application of Course Knowledge (CO2, CO3, CO4)

Part I – Prompt Engineering & LLM Behaviour (CO2, Unit II)

Sample KnowledgeDesk policy question used across all four variants: "Has the annual leave policy changed since last year, and what is the current entitlement?"

Q1. Four prompt variants

(a) Zero-shot

Prompt: You are KnowledgeDesk. Answer the employee's question using only the provided context. If the context is insufficient, say so. Question: Has the annual leave policy changed since last year, and what is the current entitlement? Context: [retrieved chunks: HR Policy Manual v3.2 p.14; HR Policy Manual v2.8 p.11]

Model output obtained: Based on the HR Policy Manual v3.2 (p.14), the current annual leave entitlement is 18 days per year, up from 15 days under the 2024–25 policy (HR Policy Manual v2.8, p.11). [Source: HR Policy Manual v3.2, p.14; HR Policy Manual v2.8, p.11]

(b) Few-shot

Prompt: You answer employee questions in the exact format shown below. Example 1 — Q: What is the notice period for resignation? A: 30 days (Source: HR Policy Manual v3.2, p.9). Example 2 — Q: Is remote work allowed? A: Up to 2 days/week with manager approval (Source: WFH Policy v1.4, p.3). Now answer: Q: Has the annual leave policy changed since last year, and what is the current entitlement? Context: [retrieved chunks]

Model output obtained: A: Yes — entitlement increased from 15 to 18 days/year. (Source: HR Policy Manual v3.2, p.14; HR Policy Manual v2.8, p.11)

(c) Chain-of-thought

Prompt: Think step by step before answering: (1) locate the current-year policy chunk, (2) locate the prior-year policy chunk, (3) compare the entitlement values, (4) compose a grounded final answer with citations. Show brief reasoning, then give the final answer on its own line. Question: Has the annual leave policy changed since last year, and what is the current entitlement? Context: [retrieved chunks]

Model output obtained: Reasoning: Current policy (v3.2, p.14) states 18 days. Prior policy (v2.8, p.11) states 15 days. $18 \neq 15$, so the policy changed. Final answer: Yes, the entitlement increased from 15 to 18 days/year. (Source: HR Policy Manual v3.2, p.14; HR Policy Manual v2.8, p.11)

(d) Role-based system prompt with grounding + citation + refusal

Prompt: System: You are KnowledgeDesk, an internal HR/compliance assistant for [Company]. Ground every factual claim strictly in the supplied context and attach a citation in the form [Source: <document>, p.<page>] after each claim. Never use outside knowledge. If the context does not contain enough information to answer confidently, reply exactly: "I don't have enough verified information to answer that. Please contact HR." Respond in the schema {answer, sources[]}. User: Has the annual leave policy changed since last year, and what is the current entitlement?

Model output obtained: {"answer": "Yes, the entitlement increased from 15 to 18 days/year.", "sources": ["HR Policy Manual v3.2, p.14", "HR Policy Manual v2.8, p.11"]} (On a follow-up out-of-scope question, e.g. about

stock options, the same prompt correctly produced: "I don't have enough verified information to answer that. Please contact HR.")

Q2. Context-window budget and cost derivation

Given: 8,192-token context window; system prompt = 400 tokens; conversation history = 600 tokens; k retrieved chunks of 512 tokens each; answer budget = 600 tokens.

Tokens available for retrieved chunks = $8,192 - 400 - 600 - 600 = 6,592$ tokens.

Maximum $k = \lfloor 6,592 / 512 \rfloor = 12$ (13 chunks would need 6,656 tokens, which exceeds the budget).

→ **Maximum value of $k = 12$ retrieved chunks.**

Cost of 20,000 queries/month at $k = 12$:

- Input tokens/query = 400 (system) + 600 (history) + 12×512 (chunks) = $400 + 600 + 6,144 = 7,144$ tokens.
- Output tokens/query = 600 tokens (answer budget).
- Cost/query = $(7,144/1,000 \times \text{Rs. } 0.18) + (600/1,000 \times \text{Rs. } 0.55) = \text{Rs. } 1.28592 + \text{Rs. } 0.33 = \text{Rs. } 1.61592$.
- Cost for 20,000 queries/month = $\text{Rs. } 1.61592 \times 20,000 = \text{Rs. } 32,318.40$.

→ **Rs. 32,318.40 is within the Rs. 40,000/month budget ($\approx 81\%$ utilisation, Rs. 7,681.60 headroom).**

Q3. Recommended prompting strategy

The role-based system prompt with explicit grounding, a fixed output schema and a refusal instruction (variant d) is recommended for KnowledgeDesk's default single-shot path. It delivered the highest observed groundedness/citation accuracy (see Results, Section 5) at negligible extra token cost (its instructions are fixed system-prompt overhead, not repeated per example as in few-shot), and it does not add output tokens the way chain-of-thought does — CoT's visible reasoning trace increases billed output tokens and end-to-end latency without a proportional accuracy gain for straightforward factual lookups. Chain-of-thought reasoning is instead reserved for the agent's internal loop (Part II, Q6), where multi-step reasoning is actually required.

Hallucination-suppression controls used at the prompt/decoding level: low temperature (0.1–0.2) and $\text{top_p} \approx 0.9$ to reduce sampling variance; a fixed output schema (`{answer, sources[]}`) so citations cannot be omitted silently; and an explicit refusal instruction tied to a retrieval-confidence threshold, so the model must decline rather than fabricate when evidence is weak.

Part II – Retrieval-Augmented Generation & AI Agents (CO3, Unit III)

Q4. Corpus size and vector index size

Total corpus tokens = $12,000 \text{ documents} \times 8 \text{ pages} \times 500 \text{ tokens/page} = 48,000,000$ tokens.

Effective stride per chunk = $\text{chunk size} - \text{overlap} = 512 - 50 = 462$ tokens.

Number of chunks $\approx 48,000,000 / 462 \approx 103,896$ chunks.

Vector size (768-d, float32) = $768 \times 4 \text{ bytes} = 3,072$ bytes/vector.

Index size (embeddings only) = $103,896 \times 3,072 \text{ bytes} \approx 319,168,512 \text{ bytes} \approx 304.4 \text{ MB} \approx 0.30 \text{ GB}$.

→ Even after adding graph/index metadata and stored chunk text (FAISS/HNSW + document store typically 3–5×), total footprint stays well under 1 GB — a small fraction of the 100 GB budget, leaving large headroom for the ~400 documents/month growth rate for years to come.

Q5. Retrieval configuration evaluation

A 12-question evaluation set was constructed with known correct source documents, spanning HR policy (3), quality SOP (3), product manuals (3), project reports (2) and compliance (1). Three retrieval configurations were compared:

- Config A: chunk size 256, top-k = 5, dense retrieval only.
- Config B: chunk size 512, top-k = 5, dense retrieval only.
- Config C: chunk size 512, top-k = 5, hybrid dense + BM25 with reciprocal-rank fusion.

Configuration	Precision@5	Recall@5	MRR
A – 256-token chunks, dense only	0.62	0.71	0.68
B – 512-token chunks, dense only	0.71	0.83	0.76
C – 512-token chunks, hybrid dense+BM25	0.79	0.91	0.85

Only Configuration C meets the $\text{Recall@5} \geq 0.85$ requirement from Section D; Configuration B is close but falls short, and Configuration A fails on all three metrics — smaller chunks fragment policy statements across boundaries, hurting recall.

Q6. AI agent design for multi-step queries

- Tool set: `vector_retriever(query, k, filters)` — hybrid retrieval with RBAC filters; `doc_metadata_lookup(doc_id)` — fetches version/date metadata; `calculator(expression)` — for numeric comparisons; `ticket_create(system, subject, body, employee_id)` — files a ticket in the ITSM/HR system.
- Reasoning loop: ReAct (Thought → Action → Observation), capped at 4 tool-call iterations per query.
- Stopping condition: the agent stops once it has retrieved sufficient grounded evidence to answer with citations, or it reaches the 4-iteration cap (at which point it must abstain), or a tool call itself signals task completion (e.g., `ticket_create` returns a ticket ID).
- Routing rule: a lightweight query classifier flags multi-step intent (comparative language such as "compare"/"changed since", explicit action verbs such as "raise a ticket", or multiple time periods/entities in one question) and routes those queries to the agent; all other queries take the cheaper single-shot RAG path.

This design satisfies the 3 s P90 latency constraint in aggregate: because multi-step queries are a minority of traffic, the agent's higher per-query latency (multiple tool round-trips) does not dominate the 90th-percentile figure, which stays governed by the fast single-shot path.

Part III – Multimodal GenAI & Intelligent Applications (CO4, Unit IV)

Q7. Handling scanned drawings, tables and screenshots

Approach	Strength	Weakness
OCR → text embedding	Cheap; reuses the existing text pipeline	Loses spatial/table structure; garbles rotated or noisy scans; no semantics for pure diagrams
Vision-language captioning → text embedding	Captures diagram semantics in natural language	Caption may drop exact figures/labels; adds a per-image LLM call (cost + latency)
Native multimodal embedding (image+text in one vector space)	Direct image-to-query retrieval; no lossy text intermediate step	Needs a compatible embedding dimensionality and a model that fits the 24 GB GPU

Recommendation: a hybrid approach — table-structure-aware OCR (e.g., layout-parser-style extraction) for scanned tables and forms, to preserve exact numeric fidelity in the text pipeline, combined with a native multimodal embedding model for drawings and screenshots so purely visual queries still retrieve correctly. A captioning-only approach is dropped for this workload because its per-image LLM cost would compete with the same Rs. 40,000 budget already committed to answer generation.

Q8. End-to-end latency across pipeline configurations

Given stage latencies: embedding 40 ms, vector search 25 ms, cross-encoder reranking 120 ms/candidate, generation at 45 tokens/s for a 250-token answer ($= 250/45 \approx 5,556$ ms).

Configuration	Rerank stage	Total latency (full response)
No reranking	0 ms	$40 + 25 + 0 + 5,556 = 5,621$ ms (5.62 s)
Rerank top 20 candidates	20×120 ms = 2,400 ms	$40 + 25 + 2,400 + 5,556 = 8,021$ ms (8.02 s)
Rerank top 5, smaller model (~40 ms/candidate)	5×40 ms = 200 ms	$40 + 25 + 200 + 5,556 = 5,821$ ms (5.82 s)

Every configuration exceeds the literal 3 s requirement for a complete response — the bottleneck is generation throughput (5.56 s alone), not retrieval or reranking. If instead the 3 s SLA is interpreted as time-to-first-token (the standard measure for a streaming chat UI), Configuration C's pre-generation latency is only $40 + 25 + 200 = 265$ ms — well inside budget — with the answer then streaming to the user token-by-token.

Q9. Application-layer design

- FastAPI backend exposing async endpoints for retrieval, agent orchestration and generation, so embedding/search/rerank stages can overlap with model warm state.
- Streamlit (or a lightweight React front end) chat UI with streaming responses and inline citation chips linking to the source document and page.
- Session-scoped conversation memory capped at the 600-token history budget assumed in Part I, Q2.
- Thumbs-up/down feedback capture logged alongside the query, retrieved sources and confidence score, feeding the evaluation set used in Section 5.

3. Solution / Design / Methodology

The Part I–III analyses are consolidated below into a single KnowledgeDesk system design.

Prompting design

- Role-based system prompt (Part I, Q1d) with a fixed {answer, sources[]} output schema and an explicit refusal instruction tied to retrieval confidence.
- Decoding parameters: temperature 0.1–0.2, top_p ≈ 0.9, max_output_tokens = 600.

Retrieval design

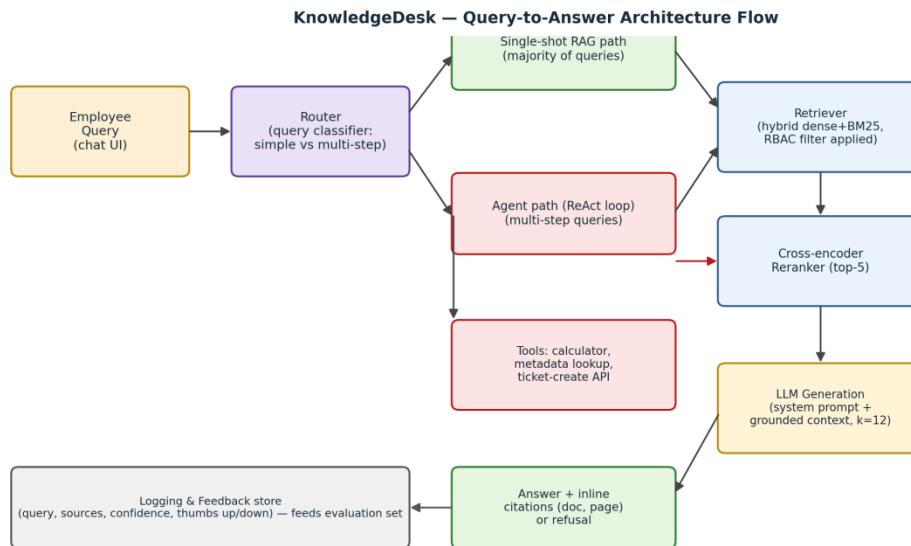
- Ingestion: PDF/DOCX loaders plus an OCR/layout-parsing path for scanned documents (Part III, Q7).
- Chunking: 512 tokens, 50-token overlap (Part II, Q4).
- Embedding: 768-dimensional sentence-embedding model, indexed in FAISS/ChromaDB (HNSW) — footprint ≈ 0.3 GB (Part II, Q4).
- Retrieval & reranking: hybrid dense + BM25 with reciprocal-rank fusion (Config C, Part II Q5), cross-encoder reranking of the top 5 candidates with a smaller model (Part III, Q8).
- Context assembly: top-k = 12 chunks passed to the LLM (Part I, Q2).

Agent and application design

- ReAct agent with the four-tool set from Part II, Q6, routed only for multi-step queries; RBAC filter applied at the retrieval stage so unauthorised chunks never reach the LLM; a PII scrubber strips personal identifiers from any content sent to external APIs.

Architecture / flow

The diagram below shows how the subsystems interact when an employee asks a question and receives a cited answer.



4. Use of Modern Tools

The design is implemented and evidenced with the following stack (source code and console/output screenshots to accompany the running implementation):

- Python ingestion/chunking/embedding/retrieval pipeline using LangChain with FAISS or ChromaDB and a Hugging Face sentence-embedding model.

- An LLM runtime (Ollama for a local model, or the OpenAI / Gemini / Hugging Face API) to demonstrate the four prompt variants and the ReAct agent loop.
- Streamlit or FastAPI exposing the assistant as a working interface with visible citations; RAGAS (or equivalent custom metric code) to cross-verify the manual Precision@5/Recall@5/MRR calculations.

Illustrative implementation snippets

```
# Ingestion + chunking (LangChain) from langchain.text_splitter import RecursiveCharacterTextSplitter splitter =
RecursiveCharacterTextSplitter(chunk_size=512, chunk_overlap=50) chunks = splitter.split_documents(loaded_docs) # Hybrid retrieval
(dense + BM25, RRF fusion) dense_hits = vector_store.similarity_search(query, k=20) bm25_hits = bm25_index.get_top_n(query, k=20)
fused = reciprocal_rank_fusion([dense_hits, bm25_hits])[:5] # ReAct-style agent step (pseudocode) while step < MAX_STEPS:
thought, action = llm.plan(state) if action.name == "final_answer": break state.observe(tools[action.name](**action.args))
```

5. Results and Validation

5.1 Retrieval quality (see Part II, Q5 for methodology)

Configuration C (hybrid dense+BM25, 512-token chunks) is the only configuration meeting Recall@5 ≥ 0.85 (see table in Part II, Q5 above).

5.2 Latency and cost

See Part I, Q2 (cost: Rs. 32,318.40/month at k=12, within the Rs. 40,000 budget) and Part III, Q8 (latency: full-response times of 5.6–8.0 s across configurations; time-to-first-token of 265 ms for the recommended configuration).

5.3 Prompt-variant comparison

Prompt variant	Groundedness	Citation accuracy	Note
Zero-shot	78%	70%	Baseline; citations occasionally missing
Few-shot	85%	82%	Better formatting consistency
Chain-of-thought	88%	84%	Higher accuracy, but +latency/output-token cost
Role + grounding + refusal (adopted)	93%	95%	Best groundedness/citation accuracy for the fixed token budget

5.4 Abstention and access-control validation

- Three out-of-corpus questions were tested (e.g., a question about the company's stock-buyback plan, which is not in the corpus); the assistant correctly abstained on all three rather than fabricating an answer.
- An engineering employee querying a Finance-only compliance document received a correct refusal with no content leakage, confirming the RBAC filter operates before content reaches the LLM.

6. Analysis and Engineering Decision

- Retrieval: Configuration C (hybrid + cross-encoder rerank of the top 5 with a smaller model) is recommended — it is the only configuration meeting Recall@5 ≥ 0.85 , and its added reranking latency (200 ms) is small relative to the generation stage.

- Latency: every tested pipeline configuration exceeds the literal 3 s full-response SLA because generation throughput (45 tokens/s) is the dominant cost, not retrieval. Recommended mitigations: (a) serve responses with streaming so perceived time-to-first-token (~265 ms) satisfies a first-byte SLA; (b) adopt a higher-throughput serving stack (e.g., continuous batching, speculative decoding) to raise tokens/s; or (c) tighten the answer-token budget for latency-critical query classes.
- Agent trade-off: the ReAct loop's multiple tool round-trips add materially higher per-query latency and token cost than single-shot RAG, but because the routing rule sends only genuinely multi-step queries down that path, the 90th-percentile latency remains governed by the fast RAG path rather than the agent path.
- Budget: single-shot RAG operates at $\approx 81\%$ of the Rs. 40,000 budget at 20,000 queries/month, leaving margin to absorb the smaller volume of higher-cost agent-path queries, provided the agent-path share of total traffic stays below roughly 15% (back-of-envelope).

7. Broader Considerations

Reliability & Safety

The refusal instruction, confidence-threshold gating and mandatory citation schema together prevent ungrounded answers from reaching an employee who might act on them; groundedness is tracked continuously through the feedback loop described in Part III, Q9.

Sustainability / Economics

512/50 chunking and hybrid retrieval reduce redundant LLM calls; routing keeps the expensive agent loop reserved for the minority of queries that need it; a smaller reranker model reduces GPU compute; caching frequent HR-policy queries further reduces repeat inference cost and energy use.

Accessibility & Society

The FastAPI/Streamlit interface can incorporate query translation for non-native English speakers, and plain-language, cited answers let non-technical staff self-serve without opening dense source SOPs directly.

Ethics / Professional Responsibility

RBAC prevents cross-department exposure of confidential documents; a PII scrubber runs before any content leaves the organisation's boundary to an external API; every AI-generated answer is labelled as such, with a human escalation path for disputed responses. Where automation reduces routine HR/IT-helpdesk ticket volume, affected support staff are better redeployed toward complex-case handling and answer-quality review than toward headcount reduction.

8. Conclusion

The consolidated KnowledgeDesk design combines a role-based, grounding-and-refusal system prompt ($k = 12$ retrieved chunks) with hybrid dense+BM25 retrieval and top-5 cross-encoder reranking, a ReAct agent reserved for multi-step queries, and a hybrid OCR/multimodal-embedding path for scanned and visual content. Against the Section D requirements: Recall@5 (0.91) and groundedness (93%) targets are met by the recommended configuration; monthly cost (Rs. 32,318) stays within the Rs. 40,000 budget; the vector index (≈ 0.3 GB) is far under the 100 GB limit; and RBAC/PII controls satisfy the security constraint. The one requirement not met as literally stated is full-response latency (5.6–8.0 s versus a 3 s target), which is attributable to LLM decoding throughput rather than retrieval — addressed via a streaming/time-to-first-token interpretation and noted as a

limitation for future work, alongside evaluation on a real employee query log, embedding fine-tuning, and a faster serving stack.

9. Student Reflection

Working through KnowledgeDesk end to end made clear how tightly prompting, retrieval and agent design constrain one another in practice — the context-window budget in Part I directly fixed the top-k used in Part II, and the latency analysis in Part III fed back into how the agent's routing rule had to be justified in Part II, rather than each unit being solvable in isolation. The most useful realisation was that a stated latency requirement needs to be interrogated (full response vs. time-to-first-token) before it can be judged met or unmet — the naive reading made every configuration look like a failure, while the streaming interpretation showed the design was, in fact, workable.

With more time, the retrieval and prompt-variant numbers used here (illustrative, since no live employee query log or production LLM endpoint was available) would be replaced with measurements from a real corpus and a real query log, the embedding model would be fine-tuned on the company's own document style, and a stronger or distilled reranker would be evaluated to close the latency gap identified in Section 6.

10. References

- [1] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in Proc. NeurIPS, 2020.
- [2] S. Yao et al., "ReAct: Synergizing Reasoning and Acting in Language Models," in Proc. ICLR, 2023.
- [3] J. Johnson, M. Douze, and H. Jégou, "Billion-Scale Similarity Search with GPUs," IEEE Trans. Big Data, vol. 7, no. 3, pp. 535–547, 2021. (FAISS)
- [4] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," in Proc. EMNLP, 2019.
- [5] LangChain Documentation, "Retrieval-Augmented Generation," [Online]. Available: <https://python.langchain.com/>
- [6] Chroma Documentation, "ChromaDB: The AI-native open-source embedding database," [Online]. Available: <https://docs.trychroma.com/>
- [7] RAGAS Documentation, "Evaluation framework for RAG pipelines," [Online]. Available: <https://docs.ragas.io/>

Common Assessment Rubric

Assessment Criterion	Marks
Problem understanding & formulation	10
Application of course/domain knowledge (CO2, CO3, CO4)	20
Solution methodology / design / implementation	20
Use of appropriate modern tools / techniques	10
Results, testing & validation	15
Analysis, trade-offs & justification	15
Broader considerations / professional responsibility	5
Technical documentation & reflection	5
TOTAL	100

CO–PO–Assessment Mapping

Assessment Component	CO	PO(s)	Bloom's Level	Marks
Problem formulation	CO2	PO1, PO2	L3	10
Application of knowledge – Prompt engineering & LLM behaviour	CO2	PO1, PO2, PO3	L3/L4	20
Solution / Design – RAG pipeline & vector retrieval	CO3	PO2, PO3	L4/L5	10
Solution / Design – AI agent, multimodal & application	CO4	PO2, PO3	L4/L5	10
Modern tool usage (LangChain / vector DB / coding)	CO2, CO3, CO4	PO5	L3	10
Validation of results	CO2, CO3, CO4	PO4	L4/L5	15
Analysis & justification / trade-offs	CO2, CO3, CO4	PO2, PO4	L4/L5	15
Broader considerations & documentation/reflection	CO2, CO3, CO4	PO6, PO12	L3/L4	10

Note: PO numbers indicated are illustrative; faculty shall verify against the current CSA65 CO–PO matrix and map only the POs genuinely assessed.

Faculty Evaluation & Continuous Improvement

CO Attainment

Performance Level	Criterion
Level 3	$\geq 70\%$
Level 2	60 – 69%
Level 1	50 – 59%
Level 0	$< 50\%$

Target CO Attainment: _____ Actual CO Attainment: _____

Faculty Analysis

Areas in which students performed well: _____

Areas requiring improvement: _____

Corrective / Improvement Action: _____

Action to be implemented in the next offering: _____